StructPaper v2 — Đề xuất kiến trúc & design pattern

Căn cứ: Yêu_cầu_phần_mềm.docx, Template_EN.docx, Template_VI.docx, DOCS_HIEN_TRANG_UPLOAD_EDITOR.md, DOCS_KIEN_TRUC_LUONG_DU_LIEU_UPLOAD_EDITOR.md và PDF mẫu ID630_HS.

0. Tóm tắt điều hành

Hiện trạng đã chạy được một đường ống trọn vẹn **Upload → CompletePaper → Editor → LaTeX → PDF**. Nhưng ba thứ đang dính chặt vào nhau nên nhiều người không thể làm song song:

1. Logic đọc DOCX nằm trong một file hơn 2.100 dòng (`docx-parser.ts`).
2. Mô hình dữ liệu quá phẳng (chuỗi text, ảnh base64, số thứ tự lưu cứng).
3. `UnifiedEditorView` điều phối mọi thứ (state, lưu, biên dịch, dàn trang).

Mọi yêu cầu lớn trong file yêu cầu (cross-reference, đánh số lại equation, renumber citation, JATS XML, nhiều journal, revision, preflight, batch) đều cần **một mô hình tài liệu có cấu trúc và ID ổn định**. Vì vậy đó là thứ phải làm đầu tiên.

Bảy quyết định lớn được đề xuất:

1. Tách **lõi miền** (`doc-model`) thuần TypeScript, không dính React, DOM, IndexedDB hay HTTP.
2. **Manuscript v2**: cây block + inline node + ID ổn định; số Figure/Table/Equation/Reference là dữ liệu *dẫn xuất*, không lưu cứng.
3. **Ingest DOCX là pipeline nhiều tầng**: ưu tiên đọc theo *style Word của template*, sau đó heuristic, cuối cùng mới AI. Chạy trong Web Worker và viết dạng isomorphic để dùng lại cho batch phía server.
4. **Journal Profile là dữ liệu** (JSON có schema), không phải code. Thêm tạp chí mới = thêm profile và một file `.cls` .
5. **Editor = Store + Command + Feature module + Block registry** (undo/redo, autosave, op-log đồng bộ BE có sẵn).
6. **Persistence offline-first**: IndexedDB (Dexie) đứng sau interface `Repository` ; sau này Spring Boot/MySQL chỉ là thêm một adapter.
7. **Compile service tách riêng**, sandbox, có cache/queue/cancel; dùng chung cho PDF preview và PDF xuất bản (PDF/X).

1. Phát hiện từ tài liệu (các bất cập cần xử lý)

#	Phát hiện	Bằng chứng trong tài liệu	Hệ quả	Hướng xử lý
1	Parser một cục	`docx-parser.ts` >2.100 dòng gồm mammoth, JSZip, OMML, metadata, bảng, ảnh, reference, nhận diện journal, merge AI	Không chia việc được; sửa bảng có thể vỡ ảnh; khó test	Pipeline nhiều tầng (mục 4)
2	Parse XML bằng regex	Mô tả dùng regex + DOM; `ommlToLatex` chỉ có 6 cấu trúc (f, sSup, sSub, sSubSup, rad, d)	Vỡ với matrix, tổng/tích phân, accent, MathType, equation array (yêu cầu mục 4 nêu rõ)	Parser XML thật; OMML → MathML → LaTeX; giữ OMML gốc
3	**Bỏ phí style của template**	Template_EN/VI có ~35 style đặt tên: ArticleTitle, Authors, Affiliation, CorrespondingAuthor, Date*, Abstract, Keywords, Heading1-3, Equation, TableCaption, TableNote, SourceLine, FigureCaption, Section*, Reference. Các style `Guide*`, `Instruction` phải bị bỏ qua	Tác giả đã dùng đúng template mà parser vẫn đoán bằng heuristic/AI; sai và tốn token	Recognizer theo style, độ tin cậy 1.0
4	Model phẳng	`content: string` chứa `[1, 2]` thô; không có inline format; `PaperSection` không có equation/list/footnote; hình/bảng không có `source`; không có block cho Ethics/Funding/Contributions/Conflict/Data availability (template có đủ)	Mất *in nghiêng* (H. pylori trong PDF mẫu), chỉ số dưới (FEV_1); không cross-ref; renumber bằng regex	Manuscript v2 (mục 3)
5	Hai biểu diễn bảng	`headers/rows` cùng `headerRows`; `normalizePaperTable` đoán header 2 tầng từ chuỗi kiểu `A / B`	Dữ liệu suy đoán, dễ sai, khó làm longtable, bảng ngang, repeat header	Một biểu diễn lưới (cell có rowSpan/colSpan)
6	Ảnh là base64 trong JSON	`src: data:...`; ép mime png/jpeg; lưu `sessionStorage` (~5 MB); `catch {}` nuốt lỗi; gửi lại base64 mỗi lần compile; alias `fig1.png/fig1.jpg`	Bài nhiều ảnh mất dữ liệu **im lặng** khi lưu; EMF/WMF/TIFF/EPS/PDF (yêu cầu) bị gán nhầm mime; không kiểm tra DPI	Asset store theo hash nội dung
7	`.doc`	UI nhận `.doc` nhưng JSZip/mammoth không đọc được `.doc` nhị phân (OLE2)	Lỗi khi người dùng chọn `.doc`	Chuyển đổi phía server (LibreOffice) hoặc từ chối rõ ràng
8	State 2 tầng + sessionStorage	`formData` ↔ `PaperContext`; `isDirty` set tay; lưu bằng Ctrl+S	Không undo/redo, không autosave, không lịch sử revision, đóng tab là mất	Store + Command + IndexedDB
9	Hai bộ sinh LaTeX	`exporters.ts` (`exportToLatex`) và `latex-template.ts` (`generateFullJournalLatex`)	Trùng lặp, dễ lệch	Một LaTeX emitter duy nhất
10	Compile ở hai nơi, chưa sandbox	Next route và Spring cùng compile; thay font bằng string replace; chạy `xelatex` trên TeX do client gửi	Rủi ro `\write18` / `\input` đọc file hệ thống; không queue/cancel/cache	Compiler service (mục 8)
11	Lộ API key, lộ bản thảo	`NEXT_PUBLIC_OPENAI_API_KEY`, người dùng gõ `sk-…`	Khóa nằm ở client; bản thảo chưa công bố đi thẳng ra ngoài	AI qua proxy BE, có consent

#	Phát hiện	Bằng chứng trong tài liệu	Hệ quả	Hướng xử lý
12	Quy tắc journal rải rác và mâu thuẫn	Tóm tắt 150–250 từ (app) vs 275–500 từ (template); từ khóa dấu phẩy vs chấm phẩy; ngày `02-10-2026` vs `dd/mm/yyyy`; HS có 8 loại bài vs template 4 loại; trích dẫn `[1]` (template) vs chỉ số trên (PDF mẫu)	Mỗi nơi một con số	Journal Profile là nguồn sự thật duy nhất
13	Song ngữ cứng	Model có `title/titleEn`; template lại tách thành 2 file EN và VI	Không mở rộng được	`lang` chính + `translations`
14	PDF mẫu có lỗi dữ liệu	DOI ghi `…v7i1.630` nhưng bài là `7(2)`	Chính là loại lỗi preflight phải bắt	Rule QC đối chiếu DOI với volume/issue; dùng PDF mẫu làm golden test thị giác

---

2. Kiến trúc tổng thể

2.1 Phân tầng (Hexagonal / Ports & Adapters)

```
apps/web  (Next.js, vỏ mỏng: route, layout, DI)
   |
packages/feature-*   UI theo tính năng (metadata, authors, body, references,
qc, layout, preview, export)
   |
packages/editor-core store, command bus, history, block registry, autosave
   |
packages/ingest-docx · exporters · qc-engine · journal-profiles · ai-client ·
storage · compiler-client
   |
packages/doc-model   types, zod schema, numbering, visitors, migrations   ←
không phụ thuộc gì

apps/api (Spring Boot 3, modular monolith)      services/compiler-worker (TeX
Live/Tectonic, sandbox)
```

Luật phụ thuộc: mũi tên chỉ đi xuống. `doc-model` không import gì; `ingest`, `exporters`, `qc` chỉ phụ thuộc `doc-model`. Ép luật bằng `dependency-cruiser` hoặc `eslint-plugin-boundaries` trong CI; vi phạm thì fail build. Đây là điều kiện để nhiều người làm song song không đạp lên nhau.

2.2 Cổng (port) và bộ chuyển đổi (adapter)

Port (interface)	Adapter giai đoạn IndexedDB	Adapter giai đoạn Spring Boot
`ManuscriptRepository`	`IndexedDbManuscriptRepo` (Dexie)	`HttpManuscriptRepo` + decorator `SyncedRepo`
`AssetRepository`	Blob trong IndexedDB	Upload qua API → S3/MinIO hoặc filesystem
`CompilerPort`	`LocalCliCompiler` (chỉ dev)	`RemoteCompiler` (job qua API)
`AiPort`	`MockAi` / bật tắt	Proxy BE → OpenAI
`JournalProfileProvider`	JSON đóng gói sẵn	API `/journals` có version
`DocConverterPort` (.doc → .docx)	không hỗ trợ, báo lỗi rõ	LibreOffice headless phía server
`Clock`, `IdGenerator`	thật	giả (để test ổn định)

UI và core chỉ biết *interface*. Đổi nơi lưu hoặc nơi biên dịch không phải sửa Editor.

2.3 Cấu trúc monorepo (pnpm workspaces + Turborepo)

```
structpaper/
├─ apps/
│  ├─ web/                  # Next.js: chỉ routing + lắp ráp (composition root)
│  └─ api/                  # Spring Boot
├─ services/
│  └─ compiler-worker/      # Docker: texlive-xetex + fonts, chạy job biên dịch
├─ packages/
│  ├─ doc-model/            # Manuscript v2, zod, numbering, visitor, migrations
│  ├─ ingest-docx/          # pipeline đọc Word (mục 4)
│  ├─ journal-profiles/     # JSON + schema + loader (mục 5)
│  ├─ exporters/            # latex, docx, jats, bibtex, zip, html
│  ├─ qc-engine/            # rule engine + rules (mục 7)
│  ├─ editor-core/          # store, commands, history, block registry
│  ├─ storage/              # repository interfaces + dexie + http + sync
```

```
|  ├─ compiler-client/        # CompilerPort + client cho API
|  ├─ ai-client/              # AiPort + prompt có version
|  ├─ feature-metadata/ feature-authors/ feature-abstract/
|  ├─ feature-body/           # heading, paragraph, figure, table, equation
|  ├─ feature-references/ feature-qc/ feature-layout/ feature-preview/
feature-export/
|  └─ ui-kit/                # Select, DatePicker, Modal, Dock...
├─ fixtures/                  # docx mẫu + JSON kỳ vọng + PDF tham chiếu
(golden)
├─ contracts/                 # openapi.yaml + JSON Schema sinh từ zod
└─ CODEOWNERS                 # chia quyền sở hữu theo package
```

---

3. Mô hình dữ liệu Manuscript v2 (`doc-model`)

3.1 Nguyên tắc

- **ID ổn định cho mọi block/inline đích tham chiếu** (ULID). Cross-reference và citation trỏ bằng ID, không bằng số.
- **Số thứ tự là dữ liệu dẫn xuất.** Figure 1, Table 2, Equation (3), `[1]` … `[n]` được tính bằng hàm `computeNumbering(doc)` từ vị trí và thứ tự xuất hiện lần đầu. Xóa/chèn giữa bài thì mọi thứ tự cập nhật theo; production editor không sửa tay (đúng yêu cầu mục 4 và 5).
- **Nội dung văn bản là cây inline**, không phải chuỗi: giữ được đậm/nghiêng/chỉ số, citation, cross-ref, công thức nội dòng.
- **Ảnh không nằm trong JSON**; chỉ có `assetId = sha256`.
- **Có `schemaVersion` + migration** (vì dữ liệu nằm lâu dài trong IndexedDB người dùng và MySQL).
- Dữ liệu nguồn gốc (provenance) được giữ lại: block nào lấy từ đâu, độ tin cậy bao nhiêu, để UI tô vàng chỗ cần người kiểm.

3.2 Hình dạng (rút gọn)

```
type Id = string; // ULID

interface Manuscript {
  schemaVersion: 2;
  id: Id;
  lang: 'vi' | 'en';                  // ngôn ngữ chính của bài (khớp
```

```
Template_VI/EN)
  journalId: string;
  articleType: string;                // lấy từ JournalProfile.articleTypes
  meta: FrontMatter;
  body: Section[];                    // cây section (tương ứng <sec> của
JATS)
  back: Statement[];                  // ethics, acknowledgments, funding,
contributions, conflict, data
  references: Reference[];            // cấu trúc hóa, không chỉ chuỗi thô
  assets: Record<string, AssetMeta>;  // sha256 -> {mime, w, h, dpi, vector}
  layoutOverrides?: LayoutOverrides;  // chỉ phần khác profile
  provenance?: Record<Id, Provenance>;
}

interface FrontMatter {
  title: Inline[]; translations?: { en?: Inline[] };
  authors: Author[];                  // {id, name, affiliationIds,
corresponding, email, orcid}
  affiliations: Affiliation[];        // {id, text, translations}
  abstract: Inline[][];               // 1 đoạn hoặc có cấu trúc
  keywords: string[];
  dates: { received?; revised?; accepted?; published? }; // ISO 8601 bên
trong, định dạng theo profile khi hiển thị
  doi?: string; volume?: string; issue?: string; pages?: [number, number];
}

type Block =
  | { type: 'paragraph'; id: Id; content: Inline[] }
  | { type: 'list'; id: Id; ordered: boolean; items: Block[][] }
  | { type: 'quote' | 'box'; id: Id; content: Block[] }
  | { type: 'equation'; id: Id; latex: string; omml?: string; numbered:
boolean; label?: string; status: 'ok' | 'fallback-image' }
  | { type: 'figure'; id: Id; assetId: string; caption: Inline[]; source?:
Inline[]; panels?: Panel[]; layout: FigureLayout }
  | { type: 'table'; id: Id; caption: Inline[]; grid: Cell[][]; headerRows:
number; notes?: Inline[]; source?: Inline[]; layout: TableLayout }
  | { type: 'pageBreak'; id: Id };

interface Section { id: Id; level: 1 | 2 | 3; title: Inline[]; blocks:
Block[]; children: Section[] }

type Inline =
  | { t: 'text'; v: string; marks?: ('b' | 'i' | 'sup' | 'sub' | 'code')[] }
```

```
  | { t: 'cite'; refIds: Id[] }                // hiển thị [1], [2-4] hoặc
chỉ số trên theo profile
  | { t: 'xref'; target: Id; kind: 'figure' | 'table' | 'equation' |
'section' }
  | { t: 'math'; latex: string }
  | { t: 'footnote'; blocks: Block[] }
  | { t: 'link'; href: string; content: Inline[] };

interface Cell { content: Inline[]; rowSpan: number; colSpan: number; align?:
'l' | 'c' | 'r'; header?: boolean }
interface Reference { id: Id; raw: string; csl?: CslJson; doi?: string; url?:
string; parse: 'ok' | 'partial' | 'failed' }
```

3.3 Dữ liệu dẫn xuất (không lưu)

```
computeNumbering(doc, citationStyle) → {
  figure: Map<Id, number>, table: Map<Id, number>, equation: Map<Id, number>,
  citationOrder: Id[],                // thứ tự xuất hiện lần đầu (kiểu đánh
số dùng làm [n], QC luôn dùng)
  citationLabels: Map<Id, string>,    // nhãn do CitationStyle tính: '3',
'2-4' hoặc 'Nguyễn, 2020'
  bibliographyOrder: Id[],            // theo thứ tự cite (Vancouver) hoặc
alphabet (APA)
  uncitedRefs: Id[], danglingCites: Id[], xrefBroken: Id[],
  firstMentionBeforeFloat: Map<Id, boolean>  // template yêu cầu nhắc trước
khi hình/bảng xuất hiện
}
```

Hàm này là *pure function*, memoize theo phiên bản document, dùng chung cho editor, QC và mọi exporter. Đây là lời giải cho ví dụ xóa Reference 5 hoặc chèn Equation giữa bài.

3.4 Di chuyển từ `CompletePaper` v1

Viết `migrateV1toV2(CompletePaper): Manuscript` (tách `[1,2]` trong chuỗi thành node `cite`, tách base64 thành asset, chuyển `headerRows` thành lưới). Giữ adapter ngược tạm thời để UI cũ vẫn chạy trong lúc chuyển (xem lộ trình, mục 14).

3.5 Hệ thống trích dẫn mở rộng được (Vancouver, APA, IEEE…)

Kiểu trích dẫn là cấu hình, không phải dữ liệu của bài. Bài chỉ lưu node `cite` trỏ tới `refIds`; cách hiển thị do một `CitationStyle` quyết định. Nhờ vậy chuyển từ Vancouver sang APA không phải sửa `Manuscript`, Editor, QC hay exporter, chỉ thêm một style.

Vancouver là kiểu đánh số theo thứ tự xuất hiện lần đầu. APA là kiểu tác giả–năm, danh mục xếp theo alphabet. Hai kiểu khác nhau ở bốn chỗ: nhãn trong bài, thứ tự danh mục, dữ liệu tối thiểu của reference và cách nhận diện khi đọc Word. Kiến trúc xử lý cả bốn chỗ như sau.

```
interface CitationStyle {
  id: string;                                    // 'vancouver' | 'apa7' |
'ieee' ...
  kind: 'numeric' | 'author-year';
  sortBibliography: 'by-first-cite' | 'alphabetical';
  formatInText(cite: CiteNode, ctx: CitationContext): Inline[];   // [2–4],
chỉ số trên, (Nguyễn và cs., 2020)
  formatBibliography(refs: Reference[], ctx: CitationContext): Block[];
}
```

Cách triển khai khuyến nghị: một `CslCitationStyle` dùng **CSL** (Citation Style Language) với `citeproc-js`. Vancouver, APA, IEEE… đều là file `.csl` có sẵn, nên thêm kiểu mới là thêm một file style và một dòng trong profile, không viết code. Profile chỉ khai báo:

```
citation: { style: 'apa7', locale: 'vi-VN', inText: 'bracket' | 'superscript'
| 'parenthetical' }
```

Tầng	Thay đổi khi thêm kiểu trích dẫn mới
`doc-model`	Node `cite` thêm `locator` (trang), `prefix`, `suffix`, `suppressAuthor` (dạng *Nguyễn (2020) cho thấy…*). `Reference.csl` trở thành dữ liệu chính, `raw` chỉ giữ bản gốc để đối chiếu
`computeNumbering`	Nhận `citationStyle`, trả thêm `citationLabels` và `bibliographyOrder`. Kiểu tác giả–năm thì danh mục theo alphabet nên không còn khái niệm renumber, nhưng `citationOrder` vẫn dùng cho QC
`journal-profiles`	`citation.style` + `locale` thay cho giá trị Vancouver cố định

Tầng	Thay đổi khi thêm kiểu trích dẫn mới
`ingest-docx`	Thêm `AuthorYearCitationRecognizer` (ghép *(Nguyễn, 2020)* với mục trong danh mục) và `ReferenceParser` (chuỗi → CSL). Nhận diện `[1]`, `[2-4]` của Vancouver vẫn giữ nguyên
`qc-engine`	Luật theo `kind`: kiểu đánh số kiểm số khớp thứ tự cite; kiểu tác giả–năm kiểm mỗi cite khớp đúng một reference (trùng tác giả–năm cần hậu tố a, b). Thêm luật *reference thiếu trường mà style mới yêu cầu* (APA cần tác giả, năm, tựa đề)
`exporters`	Nhận chuỗi đã được `citeproc` format sẵn, không để BibTeX/biblatex tự định dạng. PDF, preview HTML, DOCX và JATS cùng ra một kết quả. JATS vẫn dùng `<xref ref-type="bibr">`
Editor	Bộ chọn tài liệu tham khảo và danh mục đọc nhãn từ `citationLabels`; thêm ô *trang* và *ẩn tên tác giả* cho APA
`storage`, BE	Không đổi. Chỉ cần lưu id style trong profile (có version)

Hai lưu ý khi triển khai:

- **Dữ liệu reference quyết định khả năng đổi kiểu.** Bài Vancouver chỉ có chuỗi thô vẫn render được kiểu đánh số, nhưng không thể sang APA nếu chưa parse thành CSL. Vì vậy `ReferenceParser` (có thể kèm tra cứu DOI qua Crossref ở BE) nên làm sớm; reference có `parse: 'partial'` hoặc `'failed'` phải được QC báo trước khi đổi kiểu.
- **Ngôn ngữ ảnh hưởng định dạng.** Tên người Việt (họ đứng trước), *et al.* đổi thành *và cs.* và các chữ như *tr.* hay *Biên tập* được lấy từ `locale` của CSL, nên cần bộ locale tiếng Việt trong `citation.locale`.

3.6 Hồ sơ nhận diện bản thảo (`ManuscriptInfo`)

Mỗi bản thảo cần vài thông tin để **hệ thống và người dùng nhận ra nhanh** trong danh sách, ví dụ *Bài tổng quan loét dạ dày, bản gửi lần 2*. Đây **không phải metadata xuất bản** (tiêu đề, tác giả, DOI năm trong `meta`) và không liên quan dàn trang. Vì vậy nó được tách khỏi nội dung bài:

```
interface ManuscriptRecord {
  info: ManuscriptInfo;      // hồ sơ nhận diện: chỉ để quản lý và hiển thị
  doc: Manuscript;           // nội dung bài (mục 3.2): có rev, undo/redo,
revision
}

interface ManuscriptInfo {
  id: Id;                                // trùng doc.id
  displayName: string;                   // tên gợi nhớ, người dùng tự đặt
  submissionId?: string;                 // mã bài của tòa soạn nếu có (ví dụ
ID630_HS)
  note?: string;
  tags: string[];
  workflowStatus: 'draft' | 'in-review' | 'proofing' | 'ready' | 'published';
  pinned: boolean; archived: boolean; color?: string;
  source?: { fileName: string; importedAt: string; sha256: string };  // file
Word gốc
  ownerId?: string;
  createdAt: string; updatedAt: string; lastOpenedAt?: string;
}
```

Quy tắc:

- **Không đi vào sản phẩm xuất bản.** Exporters, QC và ingest không đọc `info`; nó không xuất hiện trong LaTeX, JATS hay PDF. Chỉ danh sách *Bản thảo của tôi*, tìm kiếm, tiêu đề tab và tên file gợi ý khi tải về dùng đến.
- **Sửa `info` không phải là sửa bài.** Đổi `displayName`, ghi chú, thẻ không tạo revision nội dung, không vào Undo/Redo, không tăng `doc.rev` và không kích hoạt biên dịch lại. Có phương thức riêng `ManuscriptRepository.updateInfo(id, patch)`.
- **Giá trị mặc định:** `displayName` lấy từ tên file Word khi tải lên (bỏ đuôi); khi trích xuất xong có thể gợi ý đổi thành tiêu đề rút gọn. Không bắt buộc duy nhất.
- **`submissionId`** dùng cho mã của tòa soạn (file PDF mẫu của bạn có dạng `ID630_HS`), tách khỏi `meta.doi`.
- **`workflowStatus`** là trạng thái quy trình, khác với điểm hay trạng thái QC. Bắt đầu với 5 giá trị tối thiểu rồi mở rộng theo quy trình tòa soạn (Editor, Author, proofing).
- **Lưu trữ:** trong Dexie, `info` là các cột phẳng của bảng `manuscripts` (có index theo `updatedAt`, `workflowStatus`, `displayName`, `tags` dạng nhiều giá trị), còn `doc` là một trường riêng. Trong MySQL là các cột `display_name`, `submission_id`,

`workflow_status`, `tags`, `pinned`, `archived`… tách khỏi `doc_json`. Màn hình danh sách chỉ đọc phần `info`, không nạp cả bài.

- **Đồng bộ:** `info` có `updated_at` riêng, xung đột xử lý kiểu *bản ghi cuối thắng*; không cần cơ chế `rev` và `If-Match` như `doc`.

---

4. Pipeline ingest DOCX (`packages/ingest-docx`)

Ý tưởng: **tách đọc (lossless) khỏi hiểu (semantic)**. Mỗi tầng là một thư mục, một nhóm người, kiểm thử độc lập.

```
.docx → [1 Package] → [2 OOXML→RawDoc] → [3 Recognizers] → [4 Assemblers] → [5 Normalizers] → [6 Validate] → Manuscript + IngestReport
```

Tầng	Việc	Pattern	Ghi chú
1. Package	Mở ZIP, đọc các part: `document.xml`, `styles.xml`, `numbering.xml`, `footnotes.xml`, `endnotes.xml`, `comments.xml`, `_rels`, `media/*`	Adapter	Chỉ phụ thuộc thư viện ZIP; trả về `DocxPackage`
2. OOXML → RawDoc	Duyệt XML **một lần** bằng parser thật (isomorphic, ví dụ `fast-xml-parser`, **không dùng DOMParser** để chạy được cả Node). Giải `basedOn` của style. Gỡ `w:sdt`, `mc:AlternateContent`. Xử lý field (`fldChar/instrText`: REF, SEQ, HYPERLINK, `ADDIN EN.CITE`, `ADDIN CSL_CITATION` của Zotero/Mendeley)	Visitor	Kết quả là `RawBlock[]` (đoạn + run + style + numbering + bảng lưới + drawing ref + math) và **chưa có ngữ nghĩa**
2b. Track Changes	Chính sách `acceptAll` (mặc định) / `rejectAll` / `keepAsComments` cho `w:ins`, `w:del`, `w:moveFrom/To`	Strategy	Đáp ứng yêu cầu Input: Word có Track Changes

Tầng	Việc	Pattern	Ghi chú
3. Recognizers	Mỗi recognizer nhìn một `RawBlock` + ngữ cảnh và trả về *claim* `{role, confidence, evidence}`	Chain of Responsibility + Strategy	Xem bên dưới
4. Assemblers	Gom các block đã gán vai trò thành từng khối miền	Builder	`FrontMatterAssembler`, `AbstractAssembler`, `BodyAssembler` (dựng cây section), `TableAssembler`, `FigureAssembler`, `EquationAssembler`, `ReferenceAssembler`, `BackmatterAssembler`, `FootnoteAssembler`
5. Normalizers	Hàm thuần `Manuscript → Manuscript`, chạy theo thứ tự	Pipeline	`UnicodeNFC` (bắt buộc cho tiếng Việt: NFD và NFC dễ lẫn), `VietnameseSpacing` (cái đang là `normalizeVietnameseWordSpacing`), `EndNoteCleanup`, `Punctuation`, `CitationLinker`, `XRefLinker`, `CaptionNumberStripper`
6. Validate	Kiểm bằng zod, sinh `IngestReport`: số block không nhận diện được, claim độ tin cậy thấp, cảnh báo	Specification	Report đi vào ngăn kéo nội dung bóc tách

4.1 Recognizer theo thứ tự ưu tiên

Ưu tiên	Recognizer	Cách làm	Độ tin cậy
1	`StyleMapRecognizer`	Dùng `JournalProfile.styleMap` (ví dụ `ArticleTitle→title`, `Authors→authors`, `Abstract→abstract`, `Heading1→h1`, `SourceLine→source`, `Reference→reference`, `Guide*` / `Instruction` → **bỏ qua**)	1.0

Ưu tiên	Recognizer	Cách làm	Độ tin cậy
2	`StructureRecognizer`	Theo vị trí và mẫu: caption `Table N.` / `Bảng N.` / `Figure N.` / `Hình N.`, `Keywords:` / `Từ khóa:`, dòng `[n]` trong danh mục, tiêu đề `References` / `Tài liệu tham khảo`	0.8
3	`FormatHeuristicRecognizer`	Cỡ chữ, in đậm, căn giữa, đoạn đầu tài liệu → title/authors	0.5–0.6
4	`AiRecognizer` (tùy chọn)	Chỉ gọi cho phần front-matter và các block còn mơ hồ (độ tin cậy < ngưỡng); trả về *đề xuất* chứ không ghi đè	0.7 + cần người duyệt

Khi hai claim xung đột, thắng bên có độ tin cậy cao hơn; mọi quyết định ghi vào `provenance` (nguồn, độ tin cậy, vị trí đoạn trong file gốc). Nhờ đó UI có thể tô cảnh báo vàng ở đúng block, và đội ingest biết chính xác recognizer nào sai.

```
interface Recognizer {
  id: string; priority: number;
  recognize(block: RawBlock, ctx: IngestContext): Claim | null;
}
// Thêm journal hay quy ước mới = thêm 1 recognizer hoặc 1 styleMap, không
đụng code cũ (Open/Closed).
```

4.2 Những điểm kỹ thuật cần giải quyết trong ingest

- **Công thức:** OMML → MathML (dùng XSLT chuẩn của Microsoft hoặc thư viện tương đương) → LaTeX; lưu cả `omml` gốc. Công thức không chuyển được thì `status: 'fallback-image'` (ảnh vector) và QC cảnh báo. Với MathType (OLE/WMF) coi là ảnh dự phòng cho tới khi có bộ chuyển.
- **Ảnh:** đọc tách riêng thành asset; nhận diện đúng định dạng bằng *magic bytes* chứ không dựa đuôi; PNG/JPEG/BMP giữ nguyên; TIFF → PNG; EMF/WMF/EPS/PDF → SVG/PDF vector (chuyển đổi phía server khi cần). Tính DPI và kích thước để QC.
- **Bảng:** dựng lưới trực tiếp từ `gridSpan` / `vMerge`, không đoán header từ chuỗi. Caption lấy từ style `TableCaption` (phía trên), `TableNote` và `SourceLine` (phía dưới).

- **Citation:** nếu có field Zotero/EndNote thì lấy dữ liệu có cấu trúc làm `Reference.csl`; nếu không, parse `[1]`, `[2-4]`, `[5,6]` thành node `cite` có `refIds`.
- **Chạy trong Web Worker** (Comlink) để không treo UI, và **viết isomorphic** để cùng package này chạy trong Node cho *batch processing* (xem mục 10).
- **`.doc` cũ:** gửi lên `DocConverterPort` (LibreOffice phía server); nếu chưa có BE thì chặn từ UI với thông báo hướng dẫn lưu thành `.docx`.

---

5. Journal Profile, dàn trang và typography

5.1 Profile là dữ liệu, không phải code

Hiện `JOURNAL_TEMPLATES` trong `templates.ts` chỉ có tên và màu, còn quy tắc khác nằm rải trong các Tab và `latex-template.ts`. Đề xuất mỗi tạp chí là một file JSON có schema (zod), có `version`, nạp qua `JournalProfileProvider`:

```
// Mọi tầng (base, journal, articleType) đều là một bản 'vá' của cùng một
hình dạng đầy đủ.
type ProfileLayer = DeepPartial<ResolvedProfile>;

interface ResolvedProfile {
  languages: ('vi' | 'en')[];
  frontMatter: {
    abstract: { required: boolean; minWords: number; maxWords: number;
structured: boolean } | null; // null = loại bài không có tóm tắt
    keywords: { min: number; max: number; separator: ';' | ',' };
    dateFormat: 'dd/MM/yyyy';
    doiPattern: string;                  // để QC đối chiếu DOI với
volume/issue
  };
  limits: { words?: number; figures?: number; tables?: number; references?:
number };
  requiredSections: string[];            // ví dụ IMRAD cho Research Article
  citation: { style: 'vancouver'; inText: 'bracket' | 'superscript';
rangeDash: '–' };
  styleMap: Record<string, string>;      // style Word -> vai trò ngữ nghĩa
(dùng ở ingest)
  layout: LayoutDefaults;                // khổ giấy, cột, lề, header/footer,
số trang, bố cục trang đầu
  typography: TypographyDefaults;        // font, cỡ chữ, leading, tracking,
giãn đoạn, hyphenation
```

```
  latex: { cls: string; engine: 'xelatex' | 'lualatex'; fonts: FontStack };
  wordTemplate?: string;                // file Word template tương ứng (nếu
có)
  qc: { ruleset: string[]; weights: Record<string, number> };
}

interface JournalProfile extends ProfileLayer {
  id: string; version: string; status: 'active' | 'draft';
  names: { vi: string; en: string; short: string }; color: string;
  articleTypes: string[];               // id loại bài; nội dung ở article-
types/<journal>/<id>.json
}

interface ArticleTypeProfile extends ProfileLayer {
  id: string; journalId: string; version: string;
  label: Record<string, string>;        // { vi: 'Bài tổng quan', en:
'Review' }
}
```

Các mâu thuẫn ở mục 1 (tóm tắt 150–250 hay 275–500 từ, từ khóa phân cách, định dạng ngày, 4 hay 8 loại bài, `[1]` hay chỉ số trên) đều được giải quyết bằng cách đặt vào profile. Khi tạp chí đổi quy định, chỉ sửa một file.

5.1b Nhiều journal, nhiều loại bài và kế thừa cấu hình

File yêu cầu nêu rõ hệ thống phải phục vụ **7 journal** của VNUHCM và không được chỉ có một template cố định:

Mã đề xuất	Journal
`std`	Science and Technology Development
`hs`	Health Sciences
`ssh`	Social Sciences and Humanities
`ebl`	Economics, Business and Law
`et`	Engineering and Technology
`ees`	Environment and Earth Sciences
`arns`	Advanced Research in Natural Sciences

Mỗi journal lại có các **loại bài (chuyên mục)** riêng. Ví dụ Health Sciences có 8 loại: Research Articles, Commentaries, Corrections, Editorials, Letters, Reviews, Methodologies, Case Reports. Trong khi đó Template_EN/VI hiện chỉ phân biệt 4 loại, nên dữ liệu loại bài phải lấy từ hướng dẫn của từng journal, không lấy từ template Word.

Ba tầng kế thừa. Nếu mỗi profile viết đủ mọi thứ thì bảy journal và hàng chục loại bài sẽ lặp lại cùng một quy tắc (Vancouver, A4, hai cột, font dự phòng) và dần lệch nhau. Vì vậy profile được tổ chức theo ba tầng, mỗi tầng chỉ ghi phần *khác* so với tầng trên:

```
base  →  journal  →  articleType
```

Tầng	Chứa gì	Ví dụ
`base`	Quy tắc chung toàn VNUHCM: Vancouver, A4, bộ `styleMap` chuẩn của Template_EN/VI, danh sách font ưu tiên và font dự phòng, bộ luật QC chung	Dùng cho cả 7 journal
`journal`	Tên, màu, ngôn ngữ, lớp `.cls`, bố cục trang đầu, định dạng ngày, từ khóa, số loại bài, danh sách loại bài	`hs`: cột trái chứa Liên hệ/Lịch sử/DOI, khung tóm tắt, chỉ số trên cho trích dẫn
`articleType`	Ghi đè theo loại bài: tóm tắt (bắt buộc hay không, có cấu trúc không, số từ), giới hạn số từ/hình/bảng/tham khảo, mục bắt buộc, bố cục trang đầu, Word template, luật QC	`hs/review`: tóm tắt không cấu trúc, không bắt buộc IMRAD; `hs/editorial`: không có tóm tắt, giới hạn ngắn

Quy tắc gộp (hàm `resolveProfile(journalId, articleTypeId, overrides)`, thuần túy và có test):

- Đối tượng gộp đệ quy; giá trị ở tầng dưới thắng.
- **Mảng ghi đè nguyên**, không nối, trừ khi khai báo rõ `$append`. Cách này tránh lỗi ngầm khi danh sách mục bắt buộc bị cộng dồn.
- Giá trị `null` nghĩa là *tắt* (ví dụ `abstract: null` cho Editorials, Letters, Corrections).
- Kết quả cuối còn được gộp thêm `manuscript.layoutOverrides` (mục 5.2).

Cấu trúc thư mục trong `packages/journal-profiles`:

```
journal-profiles/
  base.json
  journals/
    std.json  hs.json  ssh.json  ebl.json  et.json  ees.json  arns.json
  article-types/
    hs/  research.json  commentary.json  correction.json  editorial.json
         letter.json  review.json  methodology.json  case-report.json
    std/ ...   (thêm dần khi có hướng dẫn tác giả)
  word-templates/        # Template_EN/VI và template theo loại bài
  latex/                 # structpaper-base.cls + structpaper-<journal>.cls
  schema.ts              # zod: kiểm lúc build, sai là fail CI
  resolve.ts             # resolveProfile + test
```

Tác động lên các phần khác của hệ thống:

Phần	Cách dùng profile đã gộp
Ingest	Dùng `styleMap` đã gộp. Nếu Word template có style hoặc trường *Article type*, gợi ý đúng loại bài; nếu không, người dùng chọn
Editor	Chọn journal rồi chọn loại bài (danh sách lấy từ profile). Đổi loại bài giữa chừng thì QC chạy lại theo luật mới và cảnh báo mục bắt buộc còn thiếu, không xóa dữ liệu
QC	Bộ luật và ngưỡng lấy từ profile gộp, không rẽ nhánh theo tên journal trong mã
LaTeX	Một lớp gốc `structpaper-base.cls` và một lớp con cho mỗi journal. Loại bài đi qua macro (ví dụ `\spArticleType{review}`) và tùy chọn lớp, **không** tạo `.cls` riêng cho từng loại bài
Xuất Word	`DocxExporter` dùng `wordTemplate` của loại bài đã chọn
Phiên bản	`Manuscript` lưu `journalId`, `articleType` và `profileVersion`. Cập nhật quy định của tạp chí sẽ không âm thầm đổi bài đang làm; người dùng được hỏi trước khi nâng phiên bản

Phần	Cách dùng profile đã gộp
Kiểm thử	Có test ma trận *journal × loại bài*: mỗi cặp phải qua schema, gộp được, và mở được một bài mẫu mà không phát sinh lỗi cấu hình

Thứ tự triển khai.

1. Giai đoạn 1 làm đủ **cơ chế ba tầng** cùng `base` và `hs` với 8 loại bài. Hình dạng `ProfileLayer` được khóa ở Cổng 1, nên khi thêm journal không phải sửa schema.
2. Sáu journal còn lại được tạo khung từ `base` với `status: 'draft'`; giao diện hiển thị cảnh báo *quy định chưa được xác nhận* cho đến khi điền xong.
3. Dữ liệu còn thiếu: **hướng dẫn tác giả (author guidelines)** và Word template của từng journal. Trong tài liệu hiện có chỉ có thông tin của Health Sciences, nên cần tòa soạn cung cấp phần còn lại. Việc điền quy định cho mỗi journal là một task dữ liệu độc lập, làm được song song mà không đụng mã.

5.1c Lưu cấu trúc profile trong cơ sở dữ liệu

Cách tổ chức theo từng file ở mục 5.1b (một file cho `base`, mỗi journal và mỗi loại bài) được giữ làm **định dạng nguồn**. Khi có Spring Boot, cùng cấu trúc đó được nạp vào MySQL để làm nơi chạy thật. Hai nơi có vai trò khác nhau:

	Thư mục `journal-profiles/` (Git)	Cơ sở dữ liệu
Vai trò	Dữ liệu khởi tạo (seed), fixture cho test, bản sao lưu để đối chiếu	Nguồn sự thật lúc chạy, có phiên bản, bản nháp, xuất bản, audit
Ai sửa	Lập trình viên qua pull request, ở giai đoạn đầu	Production/Admin qua API (sau này có giao diện quản trị)
Công cụ	`profiles import` đọc cây thư mục, kiểm bằng zod rồi nạp vào DB	`profiles export` ghi DB ra lại cây thư mục để lưu lịch sử Git

Khi DB đã chạy thật, nơi sửa chính là DB. Việc xuất định kỳ ra Git giữ cho có lịch sử đọc được bằng `git diff`.

Lược đồ MySQL

Bảng	Cột chính	Ghi chú
`journals`	`id` (ví dụ `hs`), `code`, `status` (draft/active), `sort_order`, `created_at`	7 dòng
`article_types`	`id`, `journal_id` (FK), `code`, `sort_order`, `status`	Duy nhất theo (`journal_id`, `code`)
`profile_versions`	`id`, `scope_type` (base/journal/article_type), `scope_id` (NULL với base), `version`, `state` (draft/published/archived), `config_json` (JSON), `schema_version`, `change_note`, `created_by`, `created_at`, `published_at`	Duy nhất theo (`scope_type`, `scope_id`, `version`); mỗi scope có tối đa một bản `published`; **bản đã publish không bao giờ bị sửa**
`profile_files`	`profile_version_id`, `kind` (word_template, latex_cls, font, logo), `name`, `sha256`	Tệp thật nằm trong kho asset theo sha256, dùng chung cơ chế ở mục 10
`manuscripts` (bổ sung)	`journal_id`, `article_type_id`, `profile_pin` (JSON, ví dụ `{base:3, journal:5, articleType:2}`)	Ghim đúng phiên bản đã dùng cho bài
`audit_log` (có sẵn)	Ghi mọi lần tạo, publish, rollback profile	

Nguyên tắc vận hành

1. **Phiên bản bất biến.** Muốn đổi quy định thì tạo bản nháp mới, kiểm tra, rồi publish; không `UPDATE` bản đã publish. Rollback là publish lại nội dung của bản cũ thành một phiên bản mới.
2. **Bản thảo ghim phiên bản** (`profilePin`). Sửa quy định journal không âm thầm đổi bài đang làm. Nâng phiên bản là thao tác rõ ràng, có hiển thị khác biệt (diff) trước khi đồng ý.

3. **Kiểm tra trước khi publish.** Qua schema zod; gộp thử mọi cặp (journal, loại bài) bị ảnh hưởng; mở được bài mẫu; tùy chọn biên dịch thử một bài mẫu. Không đạt thì không cho publish.
4. **Hàm gộp (`resolveProfile`) phải cho cùng kết quả ở TypeScript và Java.** Bản TypeScript dùng ở client (làm việc ngoại tuyến) và Node worker; backend cần kết quả y hệt cho biên dịch nên cài lại cùng quy tắc (chỉ vài chục dòng) và **dùng chung một bộ test vector JSON** chạy ở cả hai phía để không lệch.
5. **Compile job nhận định danh, không nhận cấu hình.** Client gửi `(journalId, articleTypeId, profilePin)`, backend tự gộp. Nhờ vậy client không thể tự chèn cấu hình; đường dẫn `.cls` và font chỉ lấy từ `profile_files` đã được duyệt.
6. **Phân quyền.** Author và Editor chỉ đọc; Production và Admin tạo bản nháp và publish. Mọi thay đổi ghi `audit_log`.

API

Endpoint	Việc
`GET /api/v1/journals`	Danh sách journal kèm loại bài (nhẹ, cho màn hình chọn)
`GET /api/v1/profiles/resolve?journal=hs&type=review`	Trả ba lớp thô, số phiên bản và ETag (có thể thêm `pin` để lấy đúng bản đã ghim)
`GET /api/v1/profiles/{scope}/{id}/versions`	Lịch sử và diff giữa các phiên bản
`POST /api/v1/profiles/{scope}/{id}/drafts`	Tạo bản nháp
`POST /api/v1/profiles/{scope}/{id}/drafts/{v}/validate`	Chạy kiểm tra
`POST /api/v1/profiles/{scope}/{id}/drafts/{v}/publish`	Xuất bản (Admin)
`POST /api/v1/profiles/{scope}/{id}/rollback`	Quay về một phiên bản cũ
`PUT /api/v1/profiles/files/{sha256}`	Upload Word template, `.cls`, font

Phía client (offline-first)

`JournalProfileProvider` lấy dữ liệu theo thứ tự ba nguồn:

1. **IndexedDB cache** (bảng mới `profiles` với khóa `[scopeType+scopeId+version]`, và `profileIndex` giữ bản published mới nhất của mỗi scope).
2. **API**, cập nhật ngầm khi có mạng bằng `If-None-Match`.
3. **Bản đóng gói lúc build** (ảnh chụp các profile đã publish) để lần mở đầu tiên khi chưa có mạng vẫn chạy.

Bản thảo ghim phiên bản nào thì cache giữ phiên bản đó và không bị dọn khi GC. Khi chưa có backend, provider chỉ dùng nguồn 1 và 3, nên mọi thứ vẫn chạy bình thường.

Lộ trình

Giai đoạn	Việc
1	Cây thư mục file, schema, `resolveProfile`, `JournalProfileProvider` dùng bản đóng gói; interface này khóa ở Cổng 1
2	Bảng `profiles` và `profileIndex` trong Dexie
6	Bảng MySQL, API, `profiles import/export`, bộ test vector dùng chung TS và Java
7	Giao diện quản trị profile: soạn JSON có kiểm schema, diff, validate, publish, rollback

Editor và các package khác chỉ biết `JournalProfileProvider`, nên việc chuyển nguồn dữ liệu từ file sang DB không đòi hỏi sửa chúng.

5.1d Giữ lịch sử, tái hiện và nâng phiên bản profile

Mục tiêu. Khi tạp chí đổi quy định (profile có phiên bản mới), các bài đang làm hoặc đã làm xong theo quy định cũ vẫn mở lại được, vẫn sửa và kiểm tra theo đúng luật cũ, và vẫn biên dịch lại được ra kết quả giống trước. Quy định mới chỉ áp dụng cho bài khi người dùng chủ động chọn nâng phiên bản. Mục này mô tả cách làm.

1. Ba thứ phải được giữ bất biến

Ghim phiên bản của riêng cấu hình JSON là chưa đủ. Để dựng lại đúng bài cũ cần giữ cả ba thứ sau:

Thứ cần giữ	Lưu ở đâu	Quy tắc
Quy định (cấu hình đã gộp từ `base`, journal, loại bài)	`profile_versions.config_json`	Bản đã xuất bản không bao giờ bị sửa hay xóa
Tệp đi kèm (Word template, lớp `.cls`, font, logo)	Kho asset theo sha256, liên kết qua `profile_files`	Chỉ thêm, không ghi đè; tệp mới là sha256 mới
Môi trường biên dịch (phiên bản TeX Live, engine, bộ font cài trên máy chủ)	Bảng mới `compile_environments` (`id`, thẻ Docker image, phiên bản TeX Live, hash bộ font, `created_at`)	Mỗi môi trường là một dòng bất biến; image cũ được giữ lại khi còn bài dùng

Lý do cần thứ ba: cùng một profile và cùng nội dung nhưng nếu máy chủ nâng cấp TeX Live hoặc đổi font thì PDF vẫn có thể khác. Ghim quy định mà không ghim môi trường thì không tái hiện chính xác được.

2. Vòng đời một phiên bản profile

```
  tạo bản nháp ──▶ draft ──(validate đạt)──▶ validated ──(publish)──▶
published
                     ▲                                                  |
                     | sửa tiếp                                         | có
bản mới hơn được publish
                     └──── (validate không đạt)                         ▼

archived ──(phát hiện lỗi nặng)──▶ deprecated
```

Trạng thái	Ý nghĩa	Bài mới chọn được?	Bài đã ghim mở được?
`draft`	Đang soạn, chưa kiểm tra	Không	Không có bài nào ghim
`validated`	Đã qua kiểm tra, chờ xuất bản	Không	Không có bài nào ghim

Trạng thái	Ý nghĩa	Bài mới chọn được?	Bài đã ghim mở được?
`published`	Phiên bản đang hiệu lực của scope đó (tối đa một)	Có	Có
`archived`	Đã bị phiên bản mới thay	Không (ẩn khỏi danh sách chọn)	**Có, bình thường**
`deprecated`	Phát hiện lỗi nghiêm trọng trong quy định	Không	Có, nhưng hiện cảnh báo và đề nghị nâng phiên bản

Quy tắc chuyển trạng thái: chỉ đi theo chiều mũi tên; không quay `published` về `draft`. Muốn sửa một bản đã xuất bản thì tạo bản nháp mới từ nó. Rollback là *xuất bản lại nội dung của bản cũ thành phiên bản mới* (số phiên bản vẫn tăng), nên lịch sử không bao giờ bị viết lại.

Không xóa cứng. `profile_versions` và `profile_files` không có thao tác `DELETE`. Điều này được ép ở tầng cơ sở dữ liệu (không cấp quyền `DELETE` / `UPDATE` trên dòng đã `published`, hoặc dùng trigger chặn), không chỉ ở tầng ứng dụng. Việc dọn asset chỉ được xóa tệp không còn được `profile_files`, ghim của bản thảo hay ảnh chụp nào tham chiếu.

3. Bản thảo ghim phiên bản

`Manuscript` có thêm trường:

```
interface ProfilePin {
  base: number;                 // phiên bản của base
  journal: number;              // phiên bản của journal
  articleType: number;          // phiên bản của loại bài
  environmentId?: string;       // môi trường biên dịch được dùng lần gần
nhất
}
// Manuscript.profilePin: ProfilePin  (nằm trong doc, nên mọi revision và
snapshot đều mang theo)
```

`profilePin` nằm trong nội dung bài chứ không nằm trong `ManuscriptInfo`, vì nó ảnh hưởng đến cách kiểm tra và dàn trang. Nhờ vậy khôi phục một revision cũ của bài sẽ tự đưa luôn phiên bản quy định cũ trở lại, không lệch giữa nội dung và luật.

Trong MySQL, cột `manuscripts.profile_pin` (JSON) là bản sao để truy vấn, ví dụ *có bao nhiêu bài còn ghim `hs` phiên bản 5*.

4. Luồng mở một bài cũ

1. Đọc `profilePin` của bài.
2. Nạp ba lớp theo đúng ba phiên bản đã ghim: từ IndexedDB cache; nếu thiếu và đang có mạng thì lấy từ API rồi cache; nếu không có mạng và không có trong cache thì báo rõ *thiếu quy định phiên bản X, cần kết nối để tải* và chỉ cho mở chế độ xem, không cho biên dịch.
3. Nếu `config_json` có `schema_version` cũ hơn schema hiện tại thì **migrate trong bộ nhớ** (không ghi đè dữ liệu gốc).
4. Gộp bằng `resolveProfile` rồi đưa cho Editor, QC, preview và biên dịch.
5. Nếu phiên bản là `deprecated` hoặc `archived` thì hiện thanh thông báo (nhẹ với `archived`, rõ hơn với `deprecated`) kèm nút *Xem thay đổi quy định mới*.

5. Nâng phiên bản (người dùng chủ động)

Không tự động nâng phiên bản hàng loạt, vì thay đổi quy định có thể làm bài vi phạm luật mới mà tác giả chưa hay biết. Luồng nâng:

1. Người dùng bấm *Cập nhật theo quy định mới*.
2. Hệ thống tính **diff** giữa cấu hình đã gộp của phiên bản đang ghim và phiên bản mới, rồi chia nhóm tác động:

Nhóm thay đổi	Ví dụ	Hệ thống làm gì
Chỉ hiển thị	Đổi font, lề, màu	Áp dụng; xem lại preview
Ảnh hưởng QC	Giới hạn tóm tắt 250 → 300 từ, thêm mục bắt buộc	Chạy lại QC và liệt kê vấn đề mới phát sinh, chưa đổi nội dung bài
Ảnh hưởng dàn trang	Đổi `.cls`, bố cục trang đầu	Biên dịch lại và cho so sánh PDF cũ với mới
Ảnh hưởng nhận diện	Đổi `styleMap`	Không đụng dữ liệu bài đã nhập; chỉ có tác dụng khi nhập lại từ file Word
Phá vỡ	Bỏ một loại bài, đổi tên mục bắt buộc	Yêu cầu người dùng xử lý trước (ví dụ chọn loại bài khác)

3. Trước khi áp dụng, **tạo một revision snapshot** của bài. Sau đó cập nhật `profilePin`, chạy QC và ghi thêm một revision loại `profile-upgrade` (kèm phiên bản cũ, phiên bản mới và diff).
4. **Hoàn tác** bằng cách khôi phục revision trước đó; vì `profilePin` nằm trong `doc`, quy định cũ quay lại cùng nội dung.
5. Quản trị viên có *báo cáo ảnh hưởng* cho mỗi lần xuất bản phiên bản mới: có bao nhiêu bài đang ghim bản cũ, theo trạng thái quy trình. Có thể gửi thông báo nhưng **không** tự động nâng.

6. Bài đã chốt

Bài ở trạng thái quy trình `ready` hoặc `published` được coi là bất biến về quy định:

- Chặn thao tác nâng phiên bản, trừ khi mở lại quy trình (chuyển trạng thái về `draft` hoặc `proofing`).
- Khi chốt, lưu một **ảnh chụp đầy đủ** cấu hình đã gộp vào bảng `manuscript_profile_snapshots` (`manuscript_id`, `resolved_json`, `hash`, `environment_id`, `created_at`, `reason`). Ảnh chụp này là bằng chứng độc lập, đọc được ngay cả khi các lớp quy định gốc bị lưu trữ hay thay đổi cách tổ chức.

7. Tái hiện đúng PDF cũ

`compile_jobs` lưu `profile_pin`, `environment_id` và `tex_hash`. Khóa cache biên dịch gồm cả ba. Biên dịch lại một bài cũ chạy đúng image Docker và phiên bản TeX Live đã ghi, nên cùng đầu vào cho cùng kết quả (trừ những phần không xác định như ngày giờ, cần tắt hoặc cố định). Chính sách lưu giữ: giữ mọi `compile_environments` còn được bài ghim hoặc còn trong ảnh chụp tham chiếu; image của môi trường không còn ai dùng được dọn sau một thời gian lưu trữ cấu hình được (ví dụ 12 tháng).

8. Khi *schema* của profile thay đổi

Khác với *quy định* thay đổi, đây là chuyện cấu trúc dữ liệu của profile (thêm hoặc đổi tên trường):

- Mỗi `config_json` có `schema_version`; `journal-profiles` giữ chuỗi `migrate(vN→vN+1)` có test với fixture cho từng đời schema.
- Migrate chạy **trong bộ nhớ khi đọc**, không ghi đè bản đã lưu, nên bản gốc luôn nguyên vẹn để đối chiếu.
- Giữ hỗ trợ đọc ít nhất hai đời schema gần nhất; trước khi bỏ một đời, chạy báo cáo xem còn phiên bản nào ở đời đó và chuyển đổi chúng bằng cách xuất bản bản mới.

9. Phía client (ngoại tuyến)

- Bản thảo ghim phiên bản nào thì cache giữ đúng phiên bản đó (và tệp liên quan nếu cần dùng ngoại tuyến), không bị dọn khi GC.
- Mỗi lần mở bài có mạng, kiểm nhanh xem bản ghim còn `published`, `archived` hay `deprecated` để hiển thị trạng thái, không tự tải về phiên bản mới.
- Bản đóng gói lúc build chỉ chứa bản `published`; bài cũ ghim bản khác sẽ lấy từ API hoặc cache.

10. Ví dụ minh họa

Thời điểm	Sự kiện	Kết quả
T1	Bài A tạo cho `hs`, loại Reviews; ghim `hs` v5 (tóm tắt tối đa 250 từ)	Bài A kiểm theo 250 từ
T2	Production xuất bản `hs` v6 (tóm tắt tối đa 300 từ, đổi bố cục trang đầu); v5 chuyển `archived`	Bài A **không đổi**: vẫn 250 từ, vẫn bố cục cũ
T3	Bài B được tạo	Ghim v6, kiểm theo 300 từ
T4	Tác giả bài A bấm *Cập nhật theo quy định mới*	Thấy diff (QC: giới hạn tăng, dàn trang: bố cục thay đổi); hệ thống tạo snapshot, nâng lên v6, ghi revision `profile-upgrade`, biên dịch lại để so sánh
T5	Tác giả thấy bố cục mới chưa ổn	Hoàn tác bằng cách khôi phục revision trước; bài A về lại v5 cùng nội dung
T6	Phát hiện lỗi nặng trong v6	Production xuất bản v7 sửa lỗi và đánh dấu v6 `deprecated`; bài B (còn ghim v6) hiện cảnh báo và nút nâng lên v7; bài A (đang ghim v5) không bị ảnh hưởng

11. Những giới hạn cần biết

- **Ghim giữ luật, không giữ nội dung.** Nội dung bài vẫn là bản mới nhất; chỉ có *quy định áp lên nội dung* là bản cũ. Muốn xem nội dung cũ thì dùng lịch sử revision.

- **Quy định và mã chương trình là hai trục độc lập.** Phiên bản ứng dụng cũng thay đổi. CI cần có bộ bài mẫu ghim các phiên bản profile cũ để mỗi lần nâng ứng dụng đều chạy lại mở, QC và biên dịch, bắt sớm lỗi làm hỏng bài cũ.
- **Chi phí lưu trữ.** Mỗi phiên bản profile chỉ là JSON nhỏ, còn tệp `.cls`, font và image biên dịch mới đáng kể; cần chính sách lưu giữ như mục 7 và theo dõi dung lượng.

12. Thay đổi lược đồ so với mục 5.1c

Mục	Bổ sung
`profile_versions.state`	Thêm giá trị `validated` và `deprecated` (danh sách đầy đủ: `draft`, `validated`, `published`, `archived`, `deprecated`)
`compile_environments`	Bảng mới (mục 1)
`compile_jobs`	Thêm `profile_pin` (JSON) và `environment_id`
`manuscripts`	`profile_pin` (đã có); thêm `profile_snapshot_hash` cho bài đã chốt
`manuscript_profile_snapshots`	Bảng mới (mục 6)
`revisions`	Thêm loại `profile-upgrade`
Dexie `profiles`	Cache theo phiên bản ghim, loại trừ khỏi GC

5.2 Hợp nhất cấu hình lúc render

```
resolved = deepMerge(base, journal, articleType, manuscript.layoutOverrides)
```

`LayoutSettingsDrawer` chỉ ghi vào `layoutOverrides` những gì khác profile, nên đổi tạp chí đích không làm mất tùy chỉnh hợp lệ và có thể bấm *Đặt lại về mặc định của tạp chí*.

5.3 Typography (đáp ứng nhóm yêu cầu Typography)

Yêu cầu	Nơi cấu hình	Cách thực thi trong LaTeX
Font, cỡ chữ, leading	`typography.fonts`, `size`, `leading`	`fontspec`, `\linespread`; có **fallback font thật** (danh sách ưu tiên), không còn thay chuỗi MinionPro→Times New Roman bằng string replace
Tracking	`typography.tracking`	`microtype` (`letterspace`)
Giãn đoạn	`paragraphSpacing`, `indent`	`\parskip`, `\parindent`
Hyphenation	`typography.hyphenation { lang, leftMin, rightMin, enabled }`	`polyglossia`/`babel` theo `Manuscript.lang`; từng đoạn có thể mang `lang` riêng (tóm tắt tiếng Anh trong bài tiếng Việt). Tiếng Việt chủ yếu âm tiết đơn nên mặc định tắt; tiếng Anh bật
Widow/orphan	`typography.widowOrphan`	`\clubpenalty`, `\widowpenalty`
Lề, 1/2 cột, A4	`layout.page`	`geometry`, `multicol`/`twocolumn`
Header/footer, running title, số trang	`layout.header/footer/pageNumbering`	`fancyhdr` (chuyển logic hiện có `pageNumbering` sang đây)

5.4 Trang đầu và lớp `.cls` do bộ phận dàn trang sở hữu

PDF mẫu ID630_HS cho thấy trang 1 là một bố cục đặc biệt: cột trái chứa Liên hệ/Lịch sử/DOI/Bản quyền/QR, khung Tóm tắt nền xanh nhạt, hộp *Trích dẫn bài báo này ở* chân trang, đầu trang lặp tên tạp chí và số trang. Những thứ này **không nên sinh bằng TypeScript**. Đề xuất:

- Mỗi tạp chí có một lớp `structpaper-hs.cls` (hoặc `.sty`) do người làm TeX sở hữu, định nghĩa các macro như `\spTitle`, `\spAuthors`, `\spAbstract`, `\spSidebar`.
- LaTeX emitter chỉ sinh *lời gọi macro* với nội dung đã escape. Đổi thiết kế trang = sửa `.cls`, không đụng TypeScript.
- Số thứ tự (Figure, Table, Equation) do `computeNumbering` cấp, không dựa vào bộ đếm LaTeX. Hai bên khớp nhau khi preview và khi xuất.

---

6. Kiến trúc Editor

6.1 Một store, một nguồn sự thật

Bỏ mô hình hai tầng `formData` ↔ `PaperContext` và `isDirty` set tay. Thay bằng **một store** (Zustand + Immer hoặc Redux Toolkit) giữ `Manuscript`. Hiệu năng gõ phím đạt được bằng selector theo ID và `React.memo` ở từng block, không phải bằng bản nháp cục bộ:

- Component đoạn văn chỉ đăng ký `selectBlock(id)`, nên gõ một đoạn không render lại cả bài.
- Ô nhập giữ state cục bộ và commit vào store sau khoảng 150 ms (debounce).
- Danh sách block dùng ảo hóa (`react-virtuoso`) vì bài tổng quan dài.
- `isDirty` là giá trị suy ra: `rev` hiện tại khác `rev` đã lưu.

6.2 Command + History (Command, Memento)

Mọi thay đổi đi qua `CommandBus`; không component nào gọi `setState` lên tài liệu trực tiếp.

```
interface Command<P = unknown> { type: string; payload: P }
// handler: (doc, payload) => { doc: Manuscript; patches: Patch[]; inverse:
Patch[] }

bus.dispatch({ type: 'block/insert', payload: { after: blockId, block } });
bus.dispatch({ type: 'reference/remove', payload: { id } });  // numbering tự
cập nhật vì là dữ liệu dẫn xuất
bus.dispatch({ type: 'table/mergeCells', payload: { id, range } });
```

Lợi ích: **undo/redo** miễn phí (dùng patch ngược của Immer), **autosave** chỉ cần lắng nghe luồng patch, **op-log** đẩy lên BE về sau, **lịch sử revision** là chuỗi patch, và mỗi command test được độc lập không cần React. Sửa tay `isDirty`, `setPaper`, `updatePaper` đều biến mất.

6.3 Feature module (cắt dọc) và Block registry

Mỗi tab trong `EditorTabNav` là một feature package, có cùng khung:

```
feature-references/
  commands/        # reference/add, remove, reorder, parseRaw
  selectors/       # danh sách có số thứ tự, trạng thái cite
  ui/              # ReferencesTab, ReferenceItem
  qc/              # luật QC riêng của tính năng
  index.ts         # export { tab, commands, qcRules }
```

Với nội dung thân bài, dùng **Block registry** để một loại block là một đơn vị độc lập xuyên các tầng:

```
registerBlock('table', {
  schema: TableBlockSchema,
  Editor: TableEditor,            // feature-body/table
  toLatex: tableToLatex,          // exporters/latex/blocks/table.ts
  toDocx: tableToDocx,
  toJats: tableToJats,
  qc: tableRules,
});
```

TypeScript buộc kiểu `Record<BlockType, Handler>` ở mỗi tầng: thêm block mới (ví dụ `equation`) mà quên viết exporter sẽ **lỗi biên dịch**, nên không còn trường hợp sót. Ba người làm ba block khác nhau không đụng nhau.

6.4 Soạn thảo văn bản có cấu trúc

Ô `textarea` hiện tại không thể giữ in nghiêng *H. pylori*, chỉ số dưới FEV_1, citation và cross-ref như những đối tượng thật. Đề xuất dùng **TipTap/ProseMirror** cho đoạn văn và tiêu đề:

- Schema ProseMirror ánh xạ 1-1 với `Inline[]`; `cite` và `xref` là *node* có thể bấm, không phải regex tô màu.
- Gõ `[` mở bộ chọn tài liệu tham khảo; chèn *Figure/Table/Equation* bằng danh sách mục tiêu.
- Dán từ Word/Excel đi qua cùng bộ chuẩn hóa của ingest.
- Công thức: preview bằng KaTeX, sửa bằng MathLive; bảng giữ modal dạng bảng tính hiện có, đọc/ghi lưới mới.
- Chi phí: đây là phần tốn công nhất của Editor, nên đặt ở giai đoạn 4 (mục 14). Có thể làm sau khi store và command đã ổn.

6.5 Preview hai tầng

Hiện chỉ có PDF bằng XeLaTeX nên mỗi lần đổi là chờ 1–2 giây và `iframe` nhấp nháy. Đề xuất:

Tầng	Nội dung	Khi nào
Preview nhanh	React render từ AST theo profile, gần giống dàn trang	Tức thì, trong lúc gõ
Preview chính xác	PDF từ compile service, hiển thị bằng `pdf.js` (giữ nguyên vị trí cuộn, đánh dấu lỗi trên trang)	Debounce 1–2 s sau khi ngừng gõ hoặc bấm biên dịch

Từ khi emitter chèn comment `%%BLOCK:<id>` vào TeX, có thể ánh xạ ngược **lỗi LaTeX → block** (bấm lỗi để nhảy tới đúng đoạn) và dùng SyncTeX cho **bấm vào PDF → nhảy tới block**.

6.6 Bus sự kiện cho các việc nền

Các việc nền đăng ký lắng nghe luồng patch thay vì được gọi tay trong `UnifiedEditorView`:

```
document patch ─┬─▶ autosave (debounce 1 s)  ─▶ Repository.save
                ├─▶ QC engine (worker, debounce, chỉ tính lại block đổi) ─▶ qcStore
                ├─▶ compile scheduler (debounce, hủy job cũ) ─▶ previewStore
                └─▶ outbox (khi có BE) ─▶ đồng bộ
```

`UnifiedEditorView` còn lại chỉ là khung bố cục: chia đôi màn hình, dock, tab. Không còn logic nghiệp vụ.

---

7. Quality Control / Preflight (rule engine)

Hiện `computePreflightCheck` chấm 100 điểm, và `docx-extractor.ts` vừa trích xuất vừa chấm. Yêu cầu phần mềm đòi preflight rộng hơn nhiều (font missing, image resolution, overset text, lỗi link, lỗi numbering). Cách làm bền vững là **mỗi luật là một đơn vị nhỏ** (Specification pattern):

```
interface Rule {
  id: string;                          // 'ref.uncited', 'fig.dpi',
'meta.doi-volume'
  severity: 'error' | 'warning' | 'info';
  stage: 'static' | 'post-compile';    // post-compile đọc log/PDF
  appliesTo(profile: JournalProfile): boolean;
  run(ctx: QcContext): Issue[];        // ctx có doc, numbering, profile,
compileReport?
}
interface Issue { ruleId; severity; message; target: { blockId?: Id; path?:
string }; fix?: Command }
```

- Profile chọn bộ luật qua `qc.ruleset` và trọng số điểm; tạp chí khác nhau có luật khác nhau mà không rẽ nhánh trong code.
- `fix` là một Command: nút *Sửa nhanh* trong `PreflightQCTab` chỉ việc dispatch.
- Chạy trong worker, debounce, memoize theo block để không tính lại cả bài.

Nhóm	Ví dụ luật	Nguồn dữ liệu
Metadata	Thiếu tiêu đề/tác giả/email liên hệ; số từ khóa; độ dài tóm tắt theo profile; **DOI không khớp volume/issue** (lỗi có thật trong PDF mẫu)	doc + profile
Cấu trúc	IMRAD theo `articleType`; mục bắt buộc thiếu (Ethics, Funding, Conflict, Data availability); style `Guide*` còn sót trong nội dung	doc
Numbering	Figure/Table nhắc sau khi đã xuất hiện, hoặc không được nhắc; xref trỏ vào block không tồn tại; thứ tự đánh số khác thứ tự nhắc lần đầu	`computeNumbering`
Reference	Cite không có reference; reference không được cite; trùng DOI; reference parse `failed`; thiếu năm/tập/trang	doc

Nhóm	Ví dụ luật	Nguồn dữ liệu
Hình	DPI thấp, kích thước quá nhỏ cho khổ cột, thiếu caption, thiếu dòng *Nguồn*	`AssetMeta`
Bảng	Bảng là ảnh; thiếu caption hoặc dòng nguồn; bảng vượt quá chiều rộng	doc
Công thức	Chuyển đổi thất bại (`fallback-image`); công thức thiếu số hoặc thiếu tham chiếu	doc
Sau biên dịch	`Overfull \hbox` (overset text), font bị thay thế, `Reference undefined` , hình thiếu, font chưa nhúng, ảnh dưới DPI	log TeX, `pdffonts` , `pdfimages` do compile worker trả về

Báo cáo ingest (mục 4) và QC dùng chung kiểu `Issue` , nên giao diện chỉ có một danh sách vấn đề duy nhất.

---

8. Exporters và Compile service

8.1 Một AST, nhiều đầu ra (Strategy + Visitor)

Tất cả đầu ra đọc cùng `Manuscript` đã qua `computeNumbering` và `resolvedProfile` :

Exporter	Đầu ra	Ghi chú
`LatexExporter`	`article.tex` + `references.bib`	Hợp nhất `exportToLatex` và `generateFullJournalLatex` . Dùng *LaTeX writer* nhỏ với hàm `escapeTex` bắt buộc cho mọi text, có golden test
`JatsExporter`	JATS XML 1.3	Đáp ứng yêu cầu Online publishing. Có sẵn nhờ cây section, `xref` , `cite` , `Reference.csl` và `lang`
`DocxExporter`	`.docx`	Dùng lại bộ style của Template_EN/VI nên Word xuất ra khớp với đầu vào

Exporter	Đầu ra	Ghi chú
`HtmlExporter`	HTML	Dùng chung với preview nhanh
`ZipPackager`	`tex + bib + images + (xml)`	Gói nộp tòa soạn hoặc Overleaf

Mỗi exporter là một thư mục với mỗi block một file (`blocks/table.ts` , `blocks/equation.ts` ...), thỏa kiểu `Record<BlockType, Handler>` (mục 6.3).

8.2 Compile service tách riêng

Loại bỏ việc biên dịch ở hai nơi. Next.js không còn route compile trong môi trường thật; `compiler-worker` là một container riêng, Spring Boot điều phối.

```
Editor ──(manifest: tex + danh sách assetId)──▶ Spring /api/v1/compile
          ◀──────────── jobId, SSE trạng thái ────────────┐
Spring ─ kiểm tra cache theo hash(tex + assets + profile) |
       ─ asset thiếu thì client mới upload (HEAD theo sha256)
       ─ đẩy job vào hàng đợi ──▶ compiler-worker ──▶ PDF + log + báo cáo preflight
```

- **Không gửi lại base64 mỗi lần.** Ảnh upload một lần theo hash; manifest chỉ liệt kê hash. Bỏ luôn hack alias `fig1.png/fig1.jpg` .
- **Cache** theo hash nội dung (client giữ thêm một cache nhỏ ở IndexedDB), **hủy job cũ** khi có job mới của cùng bản thảo, **debounce** ở client.
- **Sandbox bắt buộc** vì TeX do hệ thống sinh từ nội dung người dùng: chạy `--no-shell-escape` , `openin_any=p` , `openout_any=p` , container không có mạng, giới hạn CPU/RAM/thời gian, thư mục tạm riêng mỗi job, người dùng không root. Vẫn escape mọi text và chặn `\input` , `\write18` do người dùng gõ vào (kể cả khi dán).
- **Cổng `CompilerPort` :** dev dùng `LocalCliCompiler` (xelatex/tectonic), production dùng `RemoteCompiler` . UI không biết khác biệt.
- **Dây chuyền prepress cho PDF xuất bản:** `xelatex → Ghostscript (PDF/X hoặc PDF/A) → kiểm font nhúng (pdffonts) → kiểm DPI (pdfimages) → nhúng metadata (hyperxmp) → bookmarks` . Kết quả preflight quay về QC (stage `post-compile`).
- **Rủi ro cần quyết sớm:** PDF/X và PDF gắn thẻ truy cập (accessibility tagging) hỗ trợ tốt hơn trên LuaLaTeX so với XeLaTeX. Nếu truy cập là bắt buộc, nên làm thử LuaLaTeX trước khi dồn công vào `.cls` . Font thương mại (MinionPro, MyriadPro) cần

giấy phép cho worker; nếu chưa có, profile phải khai báo font thay thế và QC hiện cảnh báo *đang dùng font thay thế*.

---

9. Persistence offline-first với IndexedDB (giảm tải BE), sẵn sàng cho Spring Boot + MySQL

9.1 Quyết định

- **IndexedDB là nơi làm việc chính** của trình duyệt: mở bài, sửa, undo, preview đều không cần BE. Dùng **Dexie** (API gọn, migration theo version, hỗ trợ transaction và index).
- Mọi truy cập đi qua `ManuscriptRepository` và `AssetRepository` (mục 2.2). Sau này BE chỉ là một adapter bổ sung và một *decorator* đồng bộ, **không sửa Editor**.
- Bỏ `sessionStorage`: giới hạn khoảng 5 MB, mất khi đóng tab, và `try {} catch {}` hiện tại nuốt lỗi quota một cách im lặng.

9.2 Lược đồ Dexie

Bảng	Khóa / index	Nội dung	Ghi chú
`manuscripts`	`id`; index `[journalId+updatedAt]`, `workflowStatus`, `displayName`, `tags` (nhiều giá trị), `pinned`, `archived`, `syncState`	**Hồ sơ nhận diện `info`** (mục 3.6) ở các cột phẳng; trường `doc` là `Manuscript` JSON **không chứa blob** (gồm `profilePin`, mục 5.1d); cùng `rev`, `schemaVersion`, `infoUpdatedAt`	Màn hình *Bản thảo của tôi* chỉ đọc cột `info`, không nạp `doc`. Sửa `info` không đổi `rev` và không sinh revision

Bảng	Khóa / index	Nội dung	Ghi chú
`revisions`	`id`; index `[manuscriptId+rev]`	Chuỗi patch giữa các snapshot; snapshot đầy đủ mỗi N revision hoặc khi người dùng bấm *Lưu mốc*	Có các loại `content`, `milestone`, `profile-upgrade` (kèm phiên bản profile cũ, mới và diff). Đáp ứng yêu cầu Revision và proofing; khôi phục được
`assets`	`hash` (sha256)	`Blob`, `kind` (`image`, `profile-file`), `mime`, `size`, `width`, `height`, `dpi`, `vector`	Một tệp dùng ở nhiều bản thảo chỉ lưu một lần. Tệp của profile (Word template, `.cls`, font) dùng chung bảng này với `kind: 'profile-file'`
`assetRefs`	`[ownerType+ownerId+hash]`	Quan hệ bản thảo hoặc phiên bản profile với asset	Dọn rác (GC) chỉ xóa asset không còn được bản thảo, bản ghim profile hay ảnh chụp tham chiếu
`ingestSources`	`manuscriptId`	Blob `.docx` gốc và `IngestReport`	Dùng cho *Nội dung bóc tách* và nhập lại khi có bản sửa (Revision)

Bảng	Khóa / index	Nội dung	Ghi chú
`profiles`	`[scopeType+scopeId+version]`	Cấu hình profile (`config`), `schemaVersion` , `state` , `fetchedAt`	Cache các phiên bản profile từ API hoặc bản đóng gói lúc build. **Phiên bản đang được bản thảo ghim thì không bị GC** (mục 5.1d)
`profileIndex`	`[scopeType+scopeId]`	Phiên bản `published` mới nhất, `etag` , `checkedAt`	Dùng để kiểm tra nhanh và cập nhật ngầm bằng `If-None-Match`
`outbox`	`++seq` ; index `manuscriptId`	Thao tác chờ đẩy lên BE: `{target: 'doc' hoặc 'info', op, rev, patches}` , `attempts`	Chỉ dùng khi bật đồng bộ. `doc` dùng `rev` và `If-Match` ; `info` dùng bản ghi cuối thắng
`compileCache`	`hash`	PDF Blob và thời gian	Khóa hash gồm `tex_hash` , danh sách asset, `profilePin` và `environmentId` . Giới hạn LRU theo dung lượng
`settings`	`key`	Profile đã chọn gần nhất, cấu hình giao diện, `consentAi`	Không lưu API key

9.3 Bảo đảm độ bền

- **Autosave** debounce khoảng 1 giây từ luồng patch; ghi `manuscripts` và `revisions` trong **cùng một transaction** để không lệch.

- Gọi `navigator.storage.persist()` và theo dõi `navigator.storage.estimate()`. Khi gần hết quota, hiển thị cảnh báo và đề nghị xuất ZIP; bắt `QuotaExceededError` thật, không nuốt lỗi.
- **Safari/ITP** có thể xóa dữ liệu website sau thời gian không dùng, nên IndexedDB không được coi là kho lưu trữ duy nhất khi bài quan trọng. Đây chính là lý do cần BE đồng bộ, và trước đó cần nút *Xuất bản sao lưu* (`.structpaper` = zip gồm JSON và asset).
- **Nhiều tab:** dùng `Web Locks API` hoặc `BroadcastChannel` để một bản thảo chỉ được sửa ở một tab; tab khác mở chế độ chỉ đọc.
- **Migration:** mỗi lần đổi lược đồ thì tăng `db.version(n)` và có hàm `upgrade`; riêng `Manuscript` có `schemaVersion` và chuỗi `migrate(vN→vN+1)` có test với fixture.

9.4 Chiến lược đồng bộ khi có BE

`SyncedRepository` là decorator quanh `IndexedDbRepository`: ghi cục bộ trước, đưa vào `outbox`, đẩy khi có mạng.

Giai đoạn	Cơ chế	Khi nào dùng
S1	Đồng bộ theo **bản sao**: mỗi bản thảo có `rev`; lưu lên BE bằng `If-Match: <rev>` (khóa lạc quan). Xung đột thì giữ cả hai (tạo *bản xung đột*) và để người dùng chọn	Tác giả làm một mình hoặc luân phiên; đủ cho giai đoạn đầu
S2	Đồng bộ theo **chuỗi patch** (op-log) đã có sẵn từ Command	Khi cần giảm băng thông và lưu lịch sử chi tiết trên BE
S3	CRDT (Yjs gắn với ProseMirror)	Chỉ khi thật sự cần nhiều người sửa đồng thời

Khuyến nghị bắt đầu từ S1. Vì mọi thay đổi đã là Command nên nâng lên S2 không phải viết lại Editor.

---

10. Backend Spring Boot + MySQL

10.1 Cấu trúc (modular monolith, tách service khi thật cần)

```
vn.edu.vnu.structpaper
  identity/     # đăng nhập (JWT/OIDC), vai trò: Author, Editor, Production,
Admin
  manuscript/   # bản thảo, revision, quyền truy cập
  asset/        # upload theo sha256, metadata ảnh, lưu S3/MinIO hoặc
filesystem
  journal/      # JournalProfile (version), template .cls
  compile/      # job, hàng đợi, kết quả, cache
  ingest/       # chuyển .doc → .docx (LibreOffice), batch ingest bằng Node
worker
  ai/           # proxy OpenAI, ghi log sử dụng
  common/       # lỗi chuẩn, audit, cấu hình
```

Mỗi gói có controller, service, repository, DTO riêng; không gọi chéo vào repository của gói khác (qua service/interface). Kiểm bằng ArchUnit.

10.2 Lược đồ MySQL (rút gọn)

Bảng	Cột chính	Ghi chú
`users`	`id`, `email`, `name`, `role`	Vai trò: Author, Editor, Production, Admin
`journals`	`id` (ví dụ `hs`), `code`, `status` (draft, active), `sort_order`, `created_at`	7 dòng (mục 5.1b)
`article_types`	`id`, `journal_id` (FK), `code`, `sort_order`, `status`	Duy nhất theo (`journal_id`, `code`)

Bảng	Cột chính	Ghi chú
`profile_versions`	`id`, `scope_type` (base, journal, article_type), `scope_id` (NULL với base), `version`, `state` (draft, validated, published, archived, deprecated), `config_json` (JSON), `schema_version`, `change_note`, `created_by`, `created_at`, `published_at`	Thay cho bảng `journal_profiles` trước đây. Duy nhất theo (`scope_type`, `scope_id`, `version`); mỗi scope tối đa một bản `published`; bản đã publish bất biến và không bao giờ bị xóa
`profile_files`	`profile_version_id`, `kind` (word_template, latex_cls, font, logo), `name`, `sha256`	Tệp nằm trong kho asset theo sha256; chỉ thêm, không ghi đè
`compile_environments`	`id`, `docker_image`, `texlive_version`, `fonts_hash`, `created_at`, `deprecated_at`	Môi trường biên dịch bất biến, để tái hiện đúng PDF cũ (mục 5.1d)
`manuscripts`	`id` (ULID), `owner_id`, `journal_id`, `article_type_id`, **`doc_json` (JSON)**, `rev`, `schema_version`, `profile_pin` (JSON), `profile_snapshot_hash`, `title` và `doi` (bản sao từ `meta` để tìm kiếm), **các cột `info`**: `display_name`, `submission_id`, `note`, `tags` (JSON), `workflow_status`, `pinned`, `archived`, `source_file_name`, `source_sha256`, `created_at`, `updated_at`, `last_opened_at`, `info_updated_at`	`@Version` cho khóa lạc quan của `doc`. Cột `info` tách khỏi `doc_json`; sửa `info` không đổi `rev`. Index theo `owner_id`, `workflow_status`, `journal_id`, `updated_at`

Bảng	Cột chính	Ghi chú
`manuscript_profile_snapshots`	`manuscript_id`, `resolved_json`, `hash`, `environment_id`, `created_at`, `reason`	Ảnh chụp cấu hình đã gộp khi bài được chốt (mục 5.1d)
`manuscript_revisions`	`manuscript_id`, `rev`, `kind` (patch, snapshot, milestone, profile-upgrade), `payload`, `author_id`, `created_at`	Khớp với `revisions` ở IndexedDB
`assets`	`sha256` (PK), `size`, `mime`, `width`, `height`, `dpi`, `storage_key`	Dùng chung cho ảnh bài và tệp profile
`manuscript_assets`	`manuscript_id`, `sha256`	Quan hệ bản thảo với asset
`compile_jobs`	`id`, `manuscript_id`, `rev`, `tex_hash`, `profile_pin` (JSON), `environment_id`, `status`, `started_at`, `finished_at`, `pdf_key`, `log_key`, `report_json`	Hai cột `profile_pin` và `environment_id` giúp tái hiện đúng bản cũ
`batches`	`id`, `owner_id`, `status`, `total`, `done`	Batch processing (mục 10.4)
`audit_log`	`actor`, `action`, `target`, `at`	Ghi cả tạo, publish, rollback profile và nâng phiên bản của bài

`Manuscript` là một aggregate nên lưu dạng JSON trong một dòng (MySQL 8 JSON), đồng thời phi chuẩn hóa vài cột để truy vấn. Dùng **Flyway** cho migration.

10.3 Hợp đồng FE–BE

- **OpenAPI** là nguồn sự thật. Sinh client TypeScript (`openapi-typescript` hoặc Orval) và DTO Java từ cùng một file, để hai bên không lệch.
- `Manuscript` được kiểm bằng **JSON Schema sinh từ zod**; BE (thư viện json-schema-validator) từ chối tài liệu sai `schemaVersion`. Contract test chạy trong CI cả hai phía.
- Endpoint chính:

Endpoint	Việc
`GET/PUT /api/v1/manuscripts/{id}`	Đọc và ghi bản thảo (`If-Match`)
`GET /api/v1/manuscripts/{id}/revisions`	Lịch sử
`HEAD/PUT /api/v1/assets/{sha256}`	Kiểm tra và upload ảnh theo hash
`POST /api/v1/compile` rồi `GET /api/v1/compile/{jobId}` (hoặc SSE)	Biên dịch
`GET /api/v1/journals/{id}/profile`	Profile có version
`POST /api/v1/ingest/convert`	`.doc` → `.docx`
`POST /api/v1/batches`	Nạp zip nhiều `.docx` , xử lý hàng loạt
`POST /api/v1/ai/extract`	Proxy AI

10.4 Batch/API/workflow (yêu cầu Automation)

Nhờ `ingest-docx` isomorphic, chạy cùng package trong một **Node worker** nối vào hàng đợi: `POST /batches` nhận zip, mỗi `.docx` thành một job (ingest → QC → compile), trả báo cáo tổng hợp. Không viết parser lần thứ hai bằng Java.

---

11. AI: trợ lý đề xuất, không ghi đè

- **Khóa API chỉ ở BE.** Bỏ `NEXT_PUBLIC_OPENAI_API_KEY` và ô nhập `sk-…` ở client. Client gọi `AiPort` → `POST /api/v1/ai/extract` .
- **Chỉ gửi phần cần thiết**: front-matter và các block mà recognizer đánh giá mơ hồ, không gửi cả bản thảo. Giảm token, giảm rủi ro lộ bản thảo chưa công bố. Có công tắc *đồng ý gửi nội dung ra dịch vụ AI* (`consentAi`) và ghi log sử dụng.
- **Kết quả là đề xuất dạng Command** kèm độ tin cậy; giao diện hiển thị khác biệt để người dùng chấp nhận hoặc từ chối từng thay đổi. Không còn "merge đè lên dữ liệu OpenXML".
- Dùng đầu ra có cấu trúc (JSON Schema) và prompt có version trong `ai-client` ; test bằng bộ fixture, mô hình giả (`MockAi`) cho CI.
- Tên mô hình nằm trong cấu hình BE; UI không hard-code danh sách model.

---

12. Bảng design pattern (dùng ở đâu, để làm gì)

Pattern	Áp dụng	Giải quyết vấn đề
Hexagonal (Ports & Adapters)	Repository, Compiler, AI, Converter, Profile	Đổi IndexedDB ↔ Spring, local ↔ remote mà không sửa Editor
Anti-corruption layer	`ingest-docx`: kiểu OOXML không lọt ra ngoài package	Giữ lõi miền sạch
Pipeline / Chain of Responsibility	Các tầng ingest, bộ Normalizer	Mỗi tầng một người làm, thêm bước không sửa bước cũ
Strategy	Recognizer, Exporter, Compiler, chính sách Track Changes	Thay thuật toán theo ngữ cảnh
Visitor	Duyệt AST: numbering, export, QC	Một cách duyệt, nhiều xử lý
Registry / Plugin	Block registry, Journal Profile	Thêm block hoặc tạp chí không chạm code cũ (Open/Closed)
Command + Memento	Mọi thao tác Editor, undo/redo, revision	Nhật ký thay đổi duy nhất
Repository + Decorator	`SyncedRepository` bọc `IndexedDbRepository`	Thêm đồng bộ mà không đổi interface
Observer / Event bus	Autosave, QC, compile, outbox	Bỏ logic điều phối khỏi `UnifiedEditorView`
Specification	Luật QC	Luật nhỏ, ghép được, bật tắt theo profile
Builder	Assembler (ingest), LaTeX writer	Dựng cấu trúc phức tạp từng bước
Factory	`createEmptyManuscript(profile)`, tạo block	Thay `createEmptyPaper` và các hàm rải rác
Adapter	OpenAI, TeX engine, LibreOffice	Che chi tiết bên ngoài
Derived data (selector)	`computeNumbering`	Số thứ tự không bao giờ lệch

Không dùng hết cùng lúc ở ngày đầu. Ba pattern **bắt buộc có sớm** là Command, Repository và Registry (cùng Derived data); phần còn lại thêm dần theo lộ trình.

---

13. Chiến lược kiểm thử và chất lượng mã

Loại	Cách làm	Ví dụ
Golden ingest	`fixtures/*.docx` → so sánh JSON kỳ vọng (snapshot)	Template_EN, Template_VI, 8 bài mẫu hiện có; mỗi lỗi parser sau này thêm một fixture
Unit theo tầng	Đoạn XML nhỏ cho từng assembler/normalizer	`VietnameseSpacing`: `Nguy ễ n` → `Nguyễn`; NFD→NFC
Property-based	`fast-check` cho `computeNumbering`	Chèn/xóa ngẫu nhiên block rồi kiểm số liên tục, không trùng, cite khớp
Golden export	LaTeX/JATS/DOCX so với file mẫu	Có `escapeTex` cho ký tự đặc biệt, tiếng Việt
Visual regression	Biên dịch bài mẫu, `pdftoppm` + `pixelmatch` với **PDF tham chiếu** (ID630_HS)	Bắt lệch layout khi sửa `.cls`
Contract	OpenAPI + JSON Schema chạy ở cả FE và BE	Sai `schemaVersion` bị từ chối
E2E	Playwright: upload → sửa → autosave → F5 → khôi phục → xuất PDF	Kiểm đúng lỗi mất dữ liệu hiện nay
Kiến trúc	`dependency-cruiser`, ArchUnit	Vi phạm tầng thì fail CI

Quy ước chung: TypeScript `strict`, zod ở mọi biên (nhập file, đọc IndexedDB, gọi API), lỗi trả về dạng `Result` / `Issue` thay vì `catch {}` im lặng, mỗi package có README ghi trách nhiệm và API công khai.

---

14. Lộ trình chuyển đổi (strangler, không viết lại từ đầu)

Mục tiêu: sản phẩm luôn chạy được ở cuối mỗi giai đoạn.

Giai đoạn	Việc chính	Kết quả kiểm chứng
0. Đóng băng hành vi	Dựng monorepo, CI, `dependency-cruiser`. Viết *characterization test* cho parser và exporter hiện tại bằng 8 bài mẫu và 2 template	Biết chắc đang chạy thế nào trước khi đổi
1. Lõi `doc-model`	Manuscript v2, zod, `computeNumbering`, `migrateV1toV2` + adapter ngược cho UI cũ	Số thứ tự và cite/xref tính đúng; property test xanh
2. Lưu trữ	`storage` với Dexie (manuscripts, assets, revisions), autosave, trang *Bản thảo của tôi*, nút sao lưu `.structpaper`. Tách base64 thành asset	F5/đóng tab không mất dữ liệu; bài nhiều ảnh lớn vẫn lưu được
3. Ingest pipeline	Chạy song song parser cũ và mới, so kết quả. Làm `StyleMapRecognizer` trước (ăn ngay Template_EN/VI), sau đó assemblers, OMML→MathML→LaTeX, ảnh/magic bytes. Chuyển vào Web Worker	Golden test xanh; parser cũ được gỡ khi mới đạt ngang
4. Editor core	Store + CommandBus + history; chuyển từng tab sang feature module; Block registry; thay textarea bằng ProseMirror cho đoạn văn	Undo/redo; `UnifiedEditorView` không còn logic nghiệp vụ
5. QC + Exporters	Rule engine, gộp hai bộ sinh LaTeX thành một, thêm exporter JATS, tách `.cls` cho HS	Báo cáo QC thống nhất; XML JATS hợp lệ
6. Backend	Spring Boot (manuscript, asset, journal, identity), `SyncedRepository` S1, compiler-worker có sandbox và cache, AI proxy	Mở bài ở máy khác; compile an toàn
7. Production	PDF/X + prepress, post-compile preflight, proofing (comment trên PDF), batch, S2 (đồng bộ theo patch)	Đáp ứng đầy đủ file yêu cầu

Giai đoạn 3 và 4 có thể làm song song nhờ `doc-model` đã chốt ở giai đoạn 1.

---

15. Chia việc theo team (hết cảnh "dồn cục")

Nhóm	Sở hữu (CODEOWNERS)	Phụ thuộc
Model & Core	`doc-model`, `journal-profiles` (schema)	không
Ingest	`ingest-docx` (mỗi assembler/recognizer một người, ví dụ Bảng, Hình, Công thức, Reference)	`doc-model`
Editor	`editor-core`, `feature-*`	`doc-model`, `storage`, `qc-engine`
Xuất bản	`exporters`, `compiler-worker`, `.cls` cho từng tạp chí	`doc-model`
QC	`qc-engine` và luật	`doc-model`
Backend	`apps/api`, `contracts/`	OpenAPI
Hạ tầng	CI, Docker, observability	tất cả

Như vậy việc "Bảng", "Hình", "Equation", "Reference", "Hyphenation" trong file yêu cầu là các thư mục và người sở hữu khác nhau: `ingest/assemblers/table.ts`, `exporters/latex/blocks/table.ts`, `feature-body/table/`, `qc-engine/rules/table.ts`, và typography nằm trong profile cùng `.cls`. Một thay đổi ở bảng không buộc ai đọc 2.100 dòng của parser.

---

16. Ma trận phủ yêu cầu phần mềm

Yêu cầu (file yêu cầu)	Nơi đáp ứng
Input DOCX/DOC, Track Changes	`ingest-docx` (chính sách Track Changes, `.doc` qua `DocConverterPort`)
Nhận diện Heading, tác giả, tóm tắt, từ khóa, reference…	`StyleMapRecognizer` + `StructureRecognizer` + assemblers

Yêu cầu (file yêu cầu)	Nơi đáp ứng
Template riêng từng journal, nhiều loại bài	`JournalProfile` kế thừa 3 tầng `base` → journal → `articleType` cho 7 journal; mỗi loại bài ghi đè tóm tắt, giới hạn, mục bắt buộc, bố cục, `styleMap`, Word template (mục 5.1b)
Layout 1/2 cột, A4, header/footer, số trang	`profile.layout` + `layoutOverrides` + `.cls`
Typography (font, leading, tracking, hyphenation)	`profile.typography` → `fontspec`, `microtype`, `polyglossia`
Hình: nhiều định dạng, vector, multi-panel, kiểm DPI	Asset store + `figure.panels` + QC `fig.*`
Bảng: nhiều dòng/cột, ngang, qua trang, lặp header	Lưới rowSpan/colSpan + `longtable` / `sidewaystable` + `Table N (continued)`
Công thức: OMML/MathType/LaTeX, đánh số, cross-ref	Block `equation` + OMML→MathML→LaTeX + `xref`
References Vancouver, renumber, link citation	`Reference.csl`, `cite` theo ID, `computeNumbering`
Cross-reference tự cập nhật	Node `xref` theo ID
DOI/ORCID/metadata	`FrontMatter` + QC + hyperxmp trong PDF
PDF publication/prepress	Compile worker + Ghostscript/PDF/X + preflight sau biên dịch
XML/JATS	`JatsExporter`
Proofreading PDF	Bản compile có version + comment (giai đoạn 7)
Revision không dàn lại từ đầu	`revisions` + `ingestSources` + nhập lại theo `provenance`
Batch/API/workflow	Node worker dùng lại `ingest-docx`, `POST /batches`
Quality control	`qc-engine`

17. Các điểm cần bạn xác nhận

1. **Cộng tác thời gian thực** có trong phạm vi không? Nếu không, dừng ở đồng bộ S1/S2 và bỏ CRDT.
2. **PDF/X và PDF gắn thẻ truy cập có bắt buộc không?** Nếu có, nên thử LuaLaTeX sớm, trước khi viết `.cls`.
3. **Font thương mại (MinionPro, MyriadPro)** có giấy phép dùng trên máy chủ không? Nếu không, cần chọn font thay thế chính thức.
4. **Trích dẫn trong bài:** theo template là `[1]`, còn PDF mẫu là chỉ số trên. Mặc định cho từng tạp chí là gì?
5. **Mức tin cậy với template:** có bắt buộc tác giả dùng Template_EN/VI chuẩn (ingest theo style, gần như chính xác tuyệt đối) hay vẫn phải chấp nhận Word tự do (cần heuristic/AI nhiều hơn)?

Sau khi bạn chốt các điểm trên mình có thể viết tiếp đặc tả chi tiết cho `doc-model` (zod schema đầy đủ), `StyleMapRecognizer` cho Template_VI/EN, và lược đồ Dexie dưới dạng mã.